

Minor Project Report

Comparative Analysis of LLMs for KGC in Legal Domain

Submitted in partial fulfillment of the requirement for the degree of
Master of Science in Computer Science

Submitted by

Harsh Yadav	23234747026
Vishesh Goyal	23234747061

Under the Guidance of

Prof. Vasudha Bhatnagar and Dr. Vikas Kumar
Department of Computer Science, University of Delhi



Department of Computer Science
Faculty of Mathematical Sciences
University of Delhi
Academic Year 2024-2025

Declaration

This is to certify that this project report titled “**Comparative Analysis of LLMs for KGC in Legal Domain**” is being submitted to the Department of Computer Science, University of Delhi, New Delhi in partial fulfillment of the requirement for the award of the degree of M.Sc. (Computer Science). It is a record of bonafide work carried out by us under the supervision of Prof. Vasudha Bhatnagar and Dr. Vikas Kumar. The matter embodied in the declaration has not been submitted in part or full to any university or institution for the award of any degree or diploma.

Date: January 27, 2025

Harsh Yadav (23234747026)

Place: New Delhi

Vishesh Goyal (23234747061)

Certificate

This is to certify that this project report titled “**Comparative Analysis of LLMs for KGC in Legal Domain**” submitted to the Department of Computer Science, University of Delhi, in partial fulfillment of the requirement for the award of the degree of M.Sc. (Computer Science), is a research work carried out by Harsh Yadav (23234747026) and Vishesh Goyal (23234747061) under the joint supervision of Prof. Vasudha Bhatnagar and Dr. Vikas Kumar. This work has not been submitted in part or full to any other University or Institution for the award of any degree or diploma, to the best of our knowledge.

Prof. Neelima Gupta
(Head)

Prof. Vasudha Bhatnagar
(Supervisor)

Dr. Vikas Kumar
(Supervisor)

Acknowledgement

We would like to extend our profound gratitude to our supervisor Prof. Vasudha Bhatnagar and Dr. Vikas Kumar for their interest, guidance and valuable suggestions throughout this project. We feel honoured and privileged to work under their constant supervision. Without their advice, constructive ideas, positive attitude, and continuous encouragement, it would not have been possible to make such progress in the designated time frame.

Harsh Yadav
(23234747026)

Vishesh Goyal
(23234747061)

Contents

1	Introduction	6
1.1	Understanding Information Extraction	7
1.2	Understanding Knowledge Graphs	8
1.3	Domain Specific Knowledge Graphs: The Legal Domain	11
1.4	Challenges in Legal Knowledge Graph Construction	12
1.5	Objective	13
1.6	Organization of the Report	14
2	Literature Review	15
2.1	Traditional Approaches	15
2.2	Open Information Extraction (OIE)	17
2.3	Leveraging Large Language Models for OIE	18
3	Experimental Analysis	19
3.1	Experimental Setup	19
3.1.1	Purpose	19
3.1.2	Dataset(s)	19
3.1.3	Hardware and Environment	19
3.2	Methodology	20
3.3	Experiments	21
3.3.1	Triplets Generation and Evaluation of REBEL Dataset, Testing the Pipeline	21
3.3.2	Triplets Generation of the Judgement, Comparison of the LLMs	22
3.3.3	Triplets Generation of 10 Judgment Summaries	24
3.3.4	KG Construction using Neo4j	24
4	Conclusion and Future Work	27
	References	29
A	Sentence level comparison, Mistral (7b) vs LLama3.1 (8b)	32

Chapter 1

Introduction

In an age where unstructured data is experiencing exponential growth, particularly within the legal domain, there is a critical need for tools capable of converting raw text into actionable insights. Legal texts, including statutes, case laws, contracts, and regulations, are inherently complex and unstructured due to their dense language, context-dependent interpretations, and hierarchical structures. For example, understanding a judgment often involves parsing through layers of precedent cases, specific legal jargon, and intricate relationships between parties involved. This diverse nature makes traditional retrieval of information unserviceable. To bridge this gap, there is a need for domain-specific information extraction approaches, that can extract the maximum context-rich information from the text, navigating these complexities. Once, the information is extracted, there is the need of storing and organizing it in the structured manner.

One possible solution to organize such information is through simple relational databases or keyword-based search systems. While these approaches can store and retrieve information, they struggle to capture the nuanced connections and contextual relevance inherent in legal texts. For example, keyword search might return all instances of a word like “appeal”, but fail to understand whether it refers to an appeal as a procedural step, a specific case, or a general discussion. Similarly, databases excel at storing structured data, but lose strength when faced with the unstructured, interdependent nature of legal narratives.

This is where knowledge graphs (KGs) come into play. By mapping disparate pieces of information to interconnected entities, relationships, and attributes, knowledge graphs enable systems to reason, infer, and query knowledge in a way that mirrors human cognition. Applications of KGs go way beyond structured representation. In the legal world, KGs enable sophisticated contract analytics and better legal research, simplify compliance workflows, and allow semantic searching. Encodements of knowledge in machine-readable format allow it to be extracted in precedence situations, uncover previ-

ously hidden patterns, and make possible cross-jurisdictional analyses. This has become essential for making legal practice take less time and effort to provide access to insights that lie buried under pile of textual data.

Knowledge graph construction have undergone some great development over the years. The first approaches included rule-based systems, which relied heavily on extensive amounts of manual curation and did not support scaling. Machine learning introduced statistical approaches that could scale and instead provide data-driven entity and relation extraction. However, these early models struggled with domain-specific challenges, such as understanding nuanced legal jargon. Also, there was still need to provide the schema with the algorithm to produce the triplets. The rise of deep learning and transformer architectures revolutionized the field, providing sophisticated tools like pre-trained language models capable of context-aware text comprehension. They do a great job in capturing legal semantics, term disambiguation, and finding complex hierarchical relationships.

Their ability to generalize across cases and adapt to the specific legal subdomain significantly enhances the accuracy and quality of knowledge graph construction. With LLMs, we can construct robust, scalable, and domain-adaptive legal knowledge graphs that do not only improve over the prior methodologies but also elevate standards on analytics and decision-making systems. These innovations paved the way for creating domain-specific knowledge graphs, unlocking new potential across industries, particularly in the legal domain.

1.1 Understanding Information Extraction

Information extraction (IE) is a core component of natural language processing (NLP) and heart of KGs, that focuses on identifying and structuring relevant data from unstructured textual content. The goal of IE is to extract meaningful entities, relationships, and events embedded in text, that can further be stored into databases (relational, graph, etc.), for later purposes.

There are two approaches that are the most discussed for IE. Traditional information extraction approaches require a pre-defined schema, an agreed set of specific concept types, and relation types as input. They generally use domain-based supervised learning approaches. However, training data and domain specific schema are not available in majority cases like ours, where obtaining schema and dataset for legal domain is a particularly arduous task. Therefore, the alternative approach is utilizing open information extraction models which are domain independent and pretrained on general datasets thus they don't require any predefined schema.

Triplet Extraction and Classification:

Triplet extraction involves three main steps (i) Named entity recognition (ii) Relationship extraction (iii) Event extraction. **Named Entity Recognition (NER)** includes tasks entity identification and entity typing. The former task involves identifying span of entities and the latter task focuses on containerizing these identities. The second step, **Relationship Extraction (RE)** involves identifying relationship between different entities which often act as edges of knowledge graphs. The last step, **Event Extraction (EE)** aims to identify and classify the triggered word and type that most clearly represents the occurrence of an event [1]. The structure of the resulting triplet is in the form of [Subject, Predicate, Object]. One example could be, ['Faculty of Mathematical Sciences', 'FacultyOf', 'University of Delhi'], here, 'Faculty of Mathematical Sciences' is the subject, which is associated with the object 'University of Delhi', in relation as 'FacultyOf'.

Triplet classification (often referred to as Canonicalization) is a subsequent phase in the pipeline aimed at reducing ambiguity and redundancy in the created triplets. The goal is to standardize semantically similar triplets that may have been classified as distinct due to variations in the synonyms or terms used for the predicate. By normalizing these predicate terms, the process ensures consistency and better representation of the knowledge within the knowledge graph.

1.2 Understanding Knowledge Graphs

We use Knowledge Graphs as a database to store the extracted information from the previous step, in form of nodes (representing concepts) and edges (representing relationships) as shown in the Figure 1.1, in a structured way.

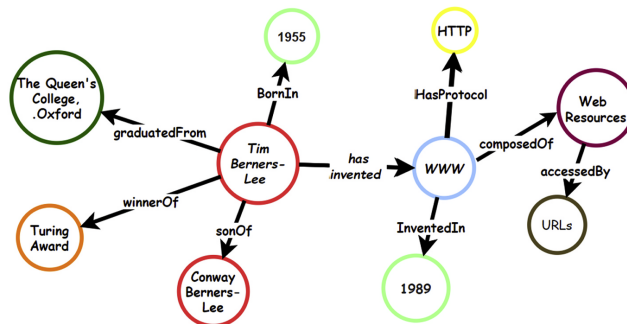


Figure 1.1: An example of entities and relations in a KG

Relational Databases vs Graph Databases

One valid question might arise here is, if we have to store the extracted information somewhere, why not to use Relational databases such as MySQL? why necessarily we

need to store it as Graphs? The answer to it is described in the paragraph below:

Knowledge graphs offer distinct advantages over relational databases in managing legal data. They naturally capture intricate relationships and hierarchies between entities, such as legal actions or obligations, which are challenging to represent with relational tables. Legal documents often contain complex, interconnected information that benefits from the flexibility of knowledge graphs, whereas relational databases rely on flat, predefined tables that require complex joins to represent such relationships, resulting in less intuitive queries. Furthermore, knowledge graphs can adapt to dynamic schemas, which is essential for the constantly evolving legal domain. This flexibility allows the addition of new entities and relationships without altering the underlying structure. In contrast, relational databases use fixed schemas, making it cumbersome to accommodate new types of legal data.

Knowledge graphs support advanced inference and reasoning by leveraging graph algorithms, allowing the deduction of legal outcomes and the exploration of connections between legal cases or statutes. They also allow more context-aware querying, enabling precise searches that identify related cases, precedents, or statutory references. Relational databases, on the other hand, lack inherent reasoning capabilities and require complex SQL queries to handle such relationships, which can be inefficient.

Broad Classification of Knowledge Graphs

Knowledge Graphs can be divided on the basis of a lot of factors like their implementation (Resource description Framework and Labeled property graphs) or construction technique (Rule based knowledge graphs and learning based knowledge graphs) but for our purposes we are dividing them on the basis of domain scope: (i) General knowledge graphs and (ii) Domain specific knowledge graphs [2]:

General Knowledge Graphs: These graphs are domain-independent and aim to organize open-world knowledge across multiple fields. Often referred to as open-world or cross-domain knowledge graphs, they aggregate information from diverse domains to provide a comprehensive view.

Some well-known examples include:

- **Google Knowledge Graph:** Powers Google Search to understand the context of search queries and present precise, structured information in response.
- **DBpedia:** Extracts structured information from Wikipedia, enabling researchers to perform large-scale semantic analysis and develop AI systems.
- **YAGO (Yet Another Great Ontology):** Combines structured data from Wikipedia

and WordNet to support linguistic and cognitive sciences research.

- **Wikidata:** Serves as a collaboratively edited central knowledge base for applications like Wikipedia and voice assistants, ensuring consistency across datasets.
- **WordNet:** Focuses on linguistic relationships between words, enabling improvements in natural language understanding and applications such as text generation.

In practical terms, these graphs support everyday technologies such as virtual assistants (like Alexa or Siri), recommendation engines, and intelligent search algorithms.

Domain-Specific Knowledge Graphs: Unlike general knowledge graphs, these graphs focus on knowledge within a specialized domain, such as biology, law, or finance. The construction of these graphs is designed to capture the intricacies and depth of the specific domain. Due to the lack of a ‘one-size-fits-all’ schema or ontology that can be applied to address real-life problems, efforts are continuing to be made to establish, polish and improve domain-specific KGs in several domains of knowledge. For example Chen et al. [3] presented K12EduKG, a domain specific KG based on K-12 educational subjects. Domain-specific educational data (Chinese curriculum standards of mathematics) was the source of knowledge that was used in K12EduKG. According to Chen et al. [3] general knowledge graphs, while effective for real-world entities like PERSON, ORGANIZATION, and LOCATION, face significant challenges when applied to the educational domain. This is because educational concepts are often abstract and cannot be easily defined or extracted, unlike the concrete entities in generic graphs. Furthermore, the relationships between educational concepts are more cognitive and implicit, making them difficult to derive from the literal meanings of the text. Similar to the above example, many other examples of different domains are listed below:

- **GeoNames:** Provides geographical data for applications like mapping and geographic information systems (GIS).
- **MusicBrainz:** Catalogs global music metadata, which powers music streaming platforms and music recommendation engines.
- **CIA World Factbook:** Organizes structured geopolitical data used for academic research and government policymaking.
- **OpenCyc:** Aims to model human common-sense reasoning and is utilized in AI systems for logical inference tasks.
- **DrugBank:** Contains detailed drug and pharmaceutical information, supporting medical research and healthcare applications.

- **Legal Knowledge Graphs:** Facilitate legal research, compliance automation, and analysis of court precedents. For example, legal KGs enable law firms to perform semantic searches through vast legal texts, trace case-law citations, and streamline contract analysis.

In this report we will only be focusing on domain specific knowledge graph construction specializing in legal domain.

1.3 Domain Specific Knowledge Graphs: The Legal Domain

Talking about the need for knowledge graph construction in legal domain, legal professionals, whether they are lawyers, judges, or legal researchers, they all rely on vast volumes of information to make informed decisions, build persuasive arguments, and navigate the intricate web of laws, regulations, and case precedents. In the era of technology-driven innovation, legal knowledge graphs are emerging as a powerful tool that promises to revolutionize the practice of law. Building a knowledge graph for the legal domain requires extracting useful information from legal documents. The structure of extracted information is mostly in the form of triplets as they are easily convertible to knowledge graphs. Extracting triplets involves same steps as described in section above.

Some of the potential applications of knowledge graphs in legal domain include [4]:

Relation Extraction and Inheritance Dispute: Inheritance disputes are the types of cases in Civil Law that focuses on the distribution of inheritance rights. Therefore, identifying the relationship between the parties is vital, as those who have the closest relationship with the deceased can get more assets. Towards this goal, relation extraction in inheritance dispute cases can provide the reason for judgment results and can improve the performance.

Contract Analysis: Contracts are unstructured documents that contain in them numerous relationships between parties, obligations, and rights. We can extract triplets from these documents for example, (**Subject**): Company A (**Predicate**): Is liable for (**Object**): Payment of damages to Company B and then construct a knowledge graph out of these triplets. By doing so, automated systems can scan and analyse these legal documents for inconsistencies, potential breaches, or clauses that require attention.

Event Timeline Extraction and Judgment Prediction of Criminal Cases: In many criminal cases, multiple parties are often involved in a group crime. To decide who should be primarily responsible for the crime, we need to determine what everyone

has done throughout the case, and the order of these events is also essential. For example, in the case of crowd fighting, the person who fights first should bear the primary responsibility. As a result, a qualified event timeline extraction model is required for predicting the outcome of criminal cases.

1.4 Challenges in Legal Knowledge Graph Construction

While knowledge graphs are highly promising for the legal domain, there are various factors that make their creation challenging:

Data privacy, stands out as a major concern related to legal data collection. Such legal documents generally house highly sensitive personally identifiable information (PII), confidential client information, or privileged data, so their building and querying must be approached with strong privacy safeguards. Generally, this would limit the availability of comprehensive datasets for training the legal knowledge graph models.

Complexity of Legal Language: Legal texts are intrinsically complex and often ambiguous, which is a major challenge for automatic extraction of entities and relationships. The highly formal language, heavy use of jargon, and the presence of multiple interpretations complicate the process of extracting accurate triplets. Specialized linguistic preprocessing and domain expertise are essential to handling these difficulties effectively.

Maintenance and Updation: Laws, regulations, and precedents keep changing. Since the legal world is dynamic in nature, updating knowledge graphs in real time, with new rulings, regulations, and other changes in the law, is difficult. Mechanisms for automated updating while maintaining integrity and relevance to the graph is a challenge.

Lack of High-Quality Triplet Datasets: A major challenge in the construction of legal knowledge graphs is the lack of high-quality, labeled triplet datasets in the legal domain. Most existing datasets are either too small or do not have sufficient coverage of various legal relationships. Moreover, these datasets lack the necessary diversity of relations needed to capture the full complexity of legal knowledge. If datasets do exist, they may be narrow in scope and fail to generalize well to the broader domain, which limits their utility for training robust legal knowledge graph models.

Super-class/Sub-class Ontologies: Many of the available legal datasets are based on super-class and sub-class ontologies, which might not always represent the rich variety of relationships found in legal texts. These ontologies often focus on hierarchical

categorizations, such as defining different types of crimes or laws, but they fall short when capturing non-hierarchical relationships like legal causality, intent, or obligations. This limits the capability of existing knowledge graphs to represent more complex and nuanced legal concepts and relationships.

Limited Domain-specific Datasets: Most of the knowledge graphs related to law often focus on very general knowledge that does not dive deep into specialized legal domains like intellectual property law, corporate law, or international treaties. Such a limitation of domain-specific datasets severely restricts the usability of knowledge graphs across niche legal domains that are heavily specialized in nature.

Despite these challenges, the potential of knowledge graphs to revolutionize the legal domain is undeniable. As NLP techniques and AI-driven reasoning improve, legal knowledge graphs will become more accurate and usable, leading to efficient legal research, contract management, and compliance. Researchers and practitioners need to continue to collaborate on overcoming these obstacles by developing new methods for legal data extraction, improving privacy-preserving mechanisms, and curating more comprehensive datasets to facilitate the construction of sophisticated, domain-specific legal knowledge graphs.

1.5 Objective

With the exponential growth of unstructured data, integrating Generative AI and Large Language Models (LLMs) with traditional approaches has become imperative essentially, standing on the shoulders of giants. Millions of legal articles, cases, and judgments are produced by courts and sub-courts worldwide. This complex, unstructured data is invaluable for legal practitioners yet often challenging to navigate effectively. Knowledge Graphs have emerged as one of the most effective ways to organize and represent such data, providing structured insights from complexity.

Our objectives for this work are as follows:

- Use the Extract-Define-Canonicalize (EDC) pipeline to **extract** triplets from legal judgments.
- **Experiment** with state-of-the-art large language models, including Mistral and Llama, and **compare** them on the bases of the generation time as well as quantity and quality of triplets using human supervision, to create a robust and effective combination tailored for the legal domain.
- **Create** a small **dataset** of high-quality triplets generated from one legal judgment

and ten judgment summaries using the above combination of pipeline and chosen LLM.

- **Create** and demonstrate a few **Knowledge Graphs** using the generated triplets to showcase their practical application.

Our work paves the way for building efficient and insightful Knowledge Graphs, aiding legal practitioners in unlocking the value hidden within complex legal texts.

1.6 Organization of the Report

The rest of the report is organized as follows: In **Chapter 2** we provide an in-depth analysis of relevant research papers that have informed and influenced the current project. In **Chapter 3**, we present our experimental setup and the analysis of the experiments using different large language models. We also report generation of triplets from 10 legal judgment summaries in this chapter. In **Chapter 4** we put forward our conclusions and possibilities of future work related to the project.

Appendix A contains the datasets used in the experiment and the triplets generated corresponding to them.

Chapter 2

Literature Review

In this chapter we present a literature review of Information Extraction(IE) systems and knowledge graph construction. Starting from simple schema based IE systems all the way to LLM based generative IE systems which are used to construct KGs.

2.1 Traditional Approaches

Traditional approaches to IE helps us in answering only very narrow well defined requests over a predefined set of target relations over a very small set of domain specific text corpus.

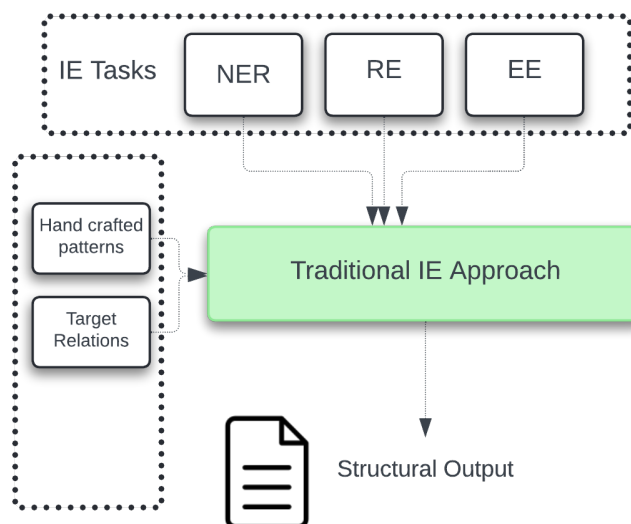


Figure 2.1: An overview of Traditional Information extraction (IE) system. It performs three broad tasks Named Entity recognition (NER), Relation Extraction (RE) and Entity Extraction (EE)

The Figure 2.1 presents an overview of how traditional information extraction works. Broadly we have to perform three tasks i.e. Named Entity Recognition (NER), Relation Extraction (RE) and Entity Extraction (EE) as shown. In traditional information extraction they are achieved using “Hand crafted patterns” and “Target Relations”.

Their input is predefined schema and hand crafted patterns learned from domain specific supervised learning. Table below shows a summary of some important IE algorithms based on traditional approaches

Name of Algorithms Feature	Learning Dictionaries for Information Extraction by Multi-Level Bootstrapping [5]	Snowball: Extracting Relations from Large Plain-Text Collections [6]	The PageRank Citation Ranking: Bringing Order to the Web [7]
Purpose	Learning dictionaries (lexicons) for information extraction from text	Extracting relations between entities from large plain-text datasets	Ranking web pages based on their importance through link analysis
Input Data	Small set of seed words or patterns, and unannotated text	Small set of seed relational tuples and large unannotated text corpora	Web pages and hyperlinks between them
Methodology	Bootstrapping to iteratively expand dictionaries based on pattern extraction	Bootstrapping and pattern-based relation extraction	Link-based ranking using the web’s hyperlink structure
Key Technique	Multi-level bootstrapping: extracting patterns and refining dictionaries iteratively	Pattern recognition and extraction of relational tuples	Graph-based algorithm: link analysis to propagate ”importance”
Application Domain	Information extraction, Named Entity Recognition (NER), domain-specific lexicons	Relation extraction (e.g., person–organization, organization–location)	Web search engines, ranking of search results
Output	A dictionary of entities, patterns, or lexical entries for information extraction	Set of relational tuples between named entities	Ranked list of web pages
Advantages	Iterative refinement with minimal supervision	Scalable to large corpora	Efficient and scalable for the web, with global importance
Limitations	Extensive human involvement; may work only for small, homogeneous corpora	May extract incorrect relations if seed or pattern is too generic	Can be manipulated (link farming); doesn’t consider content relevance

Table 2.1: Comparison of Information Extraction and Ranking Algorithms

The main drawback of these systems was that they required a lot of human

intervention and predefined domain specific schemas and datasets.

2.2 Open Information Extraction (OIE)

In 2007, the term “Open Information Extraction” was introduced by Banko et al. (2007)[8] at the University of Washington. This laid the groundwork for extracting relational tuples from text without predefined relations.

TEXTRUNNER system represents a significant advancement in OIE, in which the system makes a single, data-driven pass over the entire corpus and extracts a large set of relational tuples, without requiring human input. They also identified major challenges faced by OIE systems:

Automation: Open IE systems must automate relation extraction, which in traditional IE systems are hand labeled inputs.

Corpus Heterogeneity: Open IE systems must be able to work on heterogeneous corpora, in contrast with traditional IE systems which are only able to work on small homogeneous corpora.

Efficiency: Open IE systems must be computationally efficient in order to scale to large amounts of text.

All these criteria were first implemented in the open IE system TEXTRUNNER. This seminal work triggered a lot of research effort in this area, although not all of them end up following the initial guidelines.

History of Open Information Extraction and Knowledge Graph Construction systems:

After TEXTRUNNER [8], in 2010 WOE [9] was introduced which was a huge improvement over TEXTRUNNER in precision and F1 score. It used a novel self-supervised algorithm which led to increase in its performance. However, the weakness of WOE was that it only extracted relations mediated by verbs. This weakness was overcome by Open Language Learning for Information Extraction(OLLIE) [10] which also included relations mediated by nouns and adjective in order to improve performance. The OIE systems discussed above faced the issue of generating overly specific representations, this problem however was solved by Minimizing Facts in Open Information Extraction(MinIE) [11] . Introduced in 2017 it focused in extracting much more richer relations from text.

There are broadly two approaches used i.e. (i) discriminative and (ii) generative. In this report we won't be delving deep into these approaches rather we will provide a

general overview. The approaches defined in [12] are all discriminative, where as [1] talks about generative approaches for OIE system.

For a discriminative model, the objective is to maximize the likelihood of the data. It comprises of taking an annotated sentence x and a collection of overlapping triplets, $t_j = (s, r, o)$:

$$P_{cls}(t|x) = \prod_{(s,r,o) \in t_j} p((s, r, o)|x)$$

Where s , r and o are subject relation and object for the annotated sentence, t is collection of all the triplets. And

$$p((s, r, o)|x)$$

is the probability given sentence x probability of finding triplet consisting of (s, r, o) . Another method of discrimination involves generating tags using sequential tagging for each position i . For a sentence x consisting of n words, n different tag sequences are annotated based on the “BIESO” (Begin, Inside, End, Single, Outside) notation schema. During the training of the model, the objective is to maximize the log-likelihood of the target tag sequence by utilizing the hidden vector h_i at each position i :

$$P_{tag}(y|x) = \frac{\exp(h_i, y_i)}{\exp(\sum_{y'} \exp(h_i, y'))}$$

2.3 Leveraging Large Language Models for OIE

In recent years instead of using discriminative approach, the use of LLMs for OIE and KGC has garnered a lot of attention. The work of pretrained LLMs is to extract information particularly in form of triplets which involves identifying named entities, extracting relations and extracting events as described in chapter 1. Recent attempts to automate KGC have employed large language models (LLMs) in view of their remarkable natural language understanding capabilities.

LLM-based KGC methods employ various innovative prompt-based techniques, such as multi-turn conversation [13] and code generation [14], to generate entity-relation triplets that represent the knowledge graph. However, these methods are currently limited to small and domain-specific scenarios — to ensure the validity of generated triplets, schema information (e.g., possible entity and relation types) has to be included in the prompt. Complex datasets (e.g., Wikipedia) typically require large schemas that exceed the context window length or can be ignored by the LLMs [15].

Chapter 3

Experimental Analysis

3.1 Experimental Setup

3.1.1 Purpose

The purpose of this report is to compare and contrast the ability of copious types of pretrained LLMs along with EDC [16] on a domain specific (legal) dataset. Furthermore, using this combination of EDC and the best LLM, create triplets for 10 additional judgment summaries.

3.1.2 Dataset(s)

- **REBEL Dataset:** The original test partition of REBEL [17] comprises 105,516 entries. We selected a random sample of 1000 text-triplet pairs. This subset induces a schema with 200 distinct relation types.
- **The Judgment:** For comparing multiple LLMs, we chose a preprocessed Judgment titled “**Indian Oil Corporation v NEPC India Limited and Others - Supreme Court of India**”¹. The whole judgment comprised 366 lines/sentences.
- **10 Judgment Summaries:** The Information about filename, number of sentences in each summary is mentioned in Figure-3.4.

3.1.3 Hardware and Environment

During our experiments, we ran the EDC pipeline on the two servers simultaneously. We downloaded the repository and then setup the environment to both our local servers. The specifications of both the servers are as follows:

¹<https://indiankanoon.org/doc/39679/>

SERVER-I (System1)

Component	Details
RAM	512 GB
Processor	Intel® Xeon® Gold 6326 CPU @ 2.90 GHz (2 processors)
OS Type	64-bit operating system, x64-based processor
OS Name	Windows Server 2022 Standard
WSL Version	Ubuntu 20.04.6 LTS

Table 3.1: System Configuration Details for Server-I

SERVER-II (System2)

Component	Details
RAM	256 GB
Processor	Intel® Xeon® Gold 6226R CPU @ 2.90 GHz $\times$ 64
Graphics	NVIDIA Corporation GA102GL [RTX A6000]
OS Type	64-bit
OS Name	Ubuntu 22.04.4 LTS

Table 3.2: System Configuration Details for Server-II

Reasons for using the different servers at the same time were to compare the LLMs inference time for triplet generation on different machines, and to parallelize the process for saving time. There are few examples in our corpus that we ran on both the servers, and few that we ran on either one of them only.

3.2 Methodology

Our pipeline is based on the paper titled “Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction” [16]. In this section we explain the pipeline in detail.

Generating triplets traditionally was a complex and time-consuming task to be done by a human manually. To automate this process, [16] introduced a three-phase framework named Extract-Define-Canonicalize (EDC), which utilizes the power of pre-trained LLM’s like ChatGPT and Mistral to generate the triplets.

Additionally, [16] solves the key issue in traditional triplet generation task, i.e the compulsory need of providing a valid schema in the Large Language Model prompt to generate corresponding triplets. This can become worse if the schema is complex and exceeds the LLM’s context window length. Therefore, [16] handles this by incorporating

a technique which can work in both the scenarios; if we do have a valid schema, and we does not.

What is EDC?

EDC stands for Extract-Define-Canonicalize. These steps can be understood as:

- **Open Information Extraction (OIE)**: It is responsible for extracting the triplets from the dataset. One triplet is a pair of Subject, Relation, Object.
- **Schema Definition (SD)**: This phase is responsible for defining every component of the schema.
- **Schema Canonicalize (SC)**: This phase is responsible for eliminating the redundancy and ambiguity in the extracted triplets by utilizing the schema definitions generated above. These definitions are used to find the closest entity/relation type candidates (via a vector similarity search) that the LLM can then reference to canonicalize a component.

According to [16], the following models should be utilized corresponding to these steps:

- **OIE**: GPT-4 / GPT-3.5 Turbo / Mistral-7B-Instruct-v0.2
- **SD**: GPT-3.5 Turbo
- **SC**: GPT-3.5 Turbo & Mistral-7B-Instruct-v0.2 (for vector similarity search)

However, due to the unavailability of OPENAI KEYS, we used only open source models (Mistral-7B-Instruct / LLama 2.1-7B / LLama 3.1-8B). In all the three steps same language model has been used.

3.3 Experiments

We first ran the `example.txt` file to test the pipeline on our local system¹. Once Confirmed, we moved with REBEL dataset provided with repository itself.

3.3.1 Triplets Generation and Evaluation of REBEL Dataset, Testing the Pipeline

LLM - *Mistral-7B-Instruct-v0.2*

Initially, we extracted triplets of 19 sentences from the REBEL dataset. Followed by 100 sentences, and finally on 1000 sentences. Moving further, we ran evaluation script

to match the result of 1000 sentences with the scores claimed in the paper. But, Terminal prompted the following error on both the machines: “OOM(out of memory):Killed process issue”, where the OS automatically kills the process, if the system is under heavy memory pressure.

We suspected that the script is inefficient and has memory leaks, which is continuously increasing memory usage, and after reaching to the authors through the mail, they admitted this issue with the script.

Meantime, we discovered that tweaking the `Max_length_Diff` Parameter, can deliver the results on 1000 sentences in no-time, when setting the parameter=1, and can be set up to 4 without any fail (with increase in time for each succession). `Max_length_Diff` Parameter was skipping results that had a huge length difference with the reference. Setting this to 4 for REBEL dataset, it cut down evaluation time to ~5 mins and ~50 entries are skipped out of 1000. But, [16] did not skip any results during evaluation, and that’s the reason, it took them 3 days to evaluate 1000 sentences with 1024 GB RAM server. Our results were coming to be ~0.17, which was too low in compared to what [16] was claiming. We figured out two root causes for this mismatch:

1. Using Mistral-7B-Instruct-v0.2 in all the three steps, instead of GPT3.5 Turbo in SD and SC.
2. We were using `Max_length_Diff` parameter, which authors was not.

3.3.2 Triplets Generation of the Judgement, Comparison of the LLMs

Considering these things, we moved forward to our goal of utilizing this pipeline for Legal purposes. Our project outline was to select a Legal document (Judgment in our case), compare it with several LLMs on the following bases:

- Inference Time (monitored)
- Quality of Triplets generated (human supervised)

and finally choose the model that has the lowest triplet generation time and generates the best quality triplets among all. Evaluation with ground triplets was not done, due to unavailability of such triplets corresponding to our Indian Legal Judgment’s dataset.

Comparison

In Table 3.3, three models are listed for comparison - Mistral, LLama 2, LLama 3.1. We have experimented on LLama 3.1 with two different methods at the canonicalization

Models	Time (mins)			Total time (mins)	Count of triplets	Invalid	Valid
	OIE	SD	SC				
Mistral-7B-Instruct-v0.2	315	495	794	1604	600	365	235
LLama2-7b-chat-hf	211	437	703	1351	165	93	72
LLama3.1-8b-Instruct (Target Alignment)	210	300	65	575	70	36	34
LLama3.1-8b-Instruct (Self-Canonicalization)	200	280	52	532	866	200	666

Table 3.3: Comparison of 3 popular LLMs for triplet extraction, on Legal Judgment comprising 366 sentences. Comparison is based on required time and quality of triplets extracted.

phase of the pipeline, i.e., Target Alignment and Self-Canonicalization. Since, we did not have ground truths to evaluate the results, we compared the models on the basis of Generation time (monitored) and Quality of Triplets (human-supervised). Let’s look at them one by one.

Extraction Time

Second column represents the time taken for each model on each step (Open Information Extraction (OIE), Schema Definition (SD) and Schema Canonicalization (SC)) to extract the triplets. Third Column represents the total time combining of the three steps in minutes. In comparison, we found that Mistral took 1604 mins. (26.7 hrs) in total, which was huge for a document containing 366 sentences. Whereas, LLama 2 took 1351 mins. (22.5 h), which was slightly less than Mistral. Our third model, LLama 3.1 with Target Alignment took 575 mins. (9.6 hrs) which was a tremendous jump from the time required by LLama 2. Finally, LLama 3.1 with Self Canonicalization took 532 mins. (8.9 hrs), which was the least among all the three models we experimented.

Quality of Triplets

Fourth column represents the count of triplets extracted by each model. In comparison, we found that Mistral extracted total 600 triplets, among which 365 were marked invalid after human-supervision and only 235 were left as Valid (shown in fifth and sixth column respectively). One observation was made after analyzing the triplets that the model was hallucinating and bringing the outer context (2nd example in Table A.1) into some of the the generated triplets, which was in no sense related to the judgement. Moving on with the second model, LLama 2 extracted only 165 triplets, among which 93 were marked as invalid and remaining 72 as valid. We observed that though the generation time of this model was better than Mistral, but it was not performing better in the sense of quality, as the triplets were much sparser and contained less rich information than mistral was generating. Another issue was that it, like Mistral, was also hallucinating. Our third model, LLama 3.1 with Target Alignment extracted the least number of triplets, i.e., 70, among which 36 were marked as Invalid and 34 as Valid. The results were very poor and for the majority sentences, the model didn’t extracted even one triplet.

We tracked back in the pipeline to check the fault and found out that we were using the schema of REBEL dataset in the Canonicalization step. This form of canonicalization was named **TARGET ALIGNMENT** in [16], where we provide a schema additionally with the dataset, like the traditional triplet generation techniques we discussed in chapter-2. That’s why the models performed poorly, because the schema was not correlated with the judgment dataset and its terminologies. The [16] did provide an alternative approach for this, named **SELF CANONICALIZATION** in which it does not need to provide a schema, instead it consolidates common triplets, with the help of vector similarity and then standardizes all of them with one representation.

So with the changes this time, LLama 3.1 model with the Self-Canonicalization method extracted 866 triplets, the most number of triplets among the three, among which 200 were marked as Invalid and 666 as Valid.

We did not experimented on Mistral and LLama 2 with Self-Canonicalization method due to their extraction time, which is exceeding 20 hrs. Our aim was to select the model not solely on the basis of quality but also on the basis of time required to extract the triplets, so as to make it scalable. So, they simply extracted the triplets using Target-Alignment method.

As LLama 3.1 with Self-Canonicalization method took the least time and extracted the best quality and quantity of triplets, so we decided to use this setting in our further triplet generation task on the legal documents.

3.3.3 Triplets Generation of 10 Judgment Summaries

LLM *LLama3.1-8b-Instruct (Self-Canonicalization)*

Once the model was selected, we proceeded to our next task, which was to generate triplets corresponding to 10 judgment summaries. The Information about summary filename, no. of sentences in the corresponding summary, time taken at each step (OIE, SD, SC) on SYSTEM 1 and SYSTEM 2, and total time is provided in Table-3.4.

3.3.4 KG Construction using Neo4j

Using the triplets extracted above, we also experimented constructing the KG’s, out of curiosity. Below are some snapshots of how they look.

Filename (.txt)	No. of sentences	SYS 1's time (mins)			SYS 2's time (mins)			Total time (mins)	Count of triplets	Invalid	Valid
		OIE	SD	SC	OIE	SD	SC				
64813_ed	21	11	15	4	16	25	6	30, 47	48	8	40
64813_gd	23	13	24	4	19	38	7	41, 64	56	12	44
745161_ed	21	-	-	-	21	35	7	63	72	17	55
745161_gd	38	25	37	5	-	-	-	67	108	14	94
1114158_ed	25	-	-	-	20	37	7	64	64	19	45
1114158_gd	38	19	29	7	-	-	-	55	82	22	60
1316559_ed	55	36	50	9	-	-	-	95	173	85	88
1316559_gd	53	-	-	-	51*	77*	14*	140*	180	86	94
1363234_ed	30	18	25	6	-	-	-	50	74	25	49
1363234_gd	26	17	30	5	-	-	-	52	79	29	50

Table 3.4: Time taken by SYSTEM 1 and SYSTEM 2 for each step (OIE, SD, and SC) corresponding to each document. Count of triplets is also mentioned along with count of invalid and valid triplets based on human evaluation. (* denotes running of another script parallel on the system, impacting increase in the generation time). ed and gd in filename column stands for extracted and gold respectively.

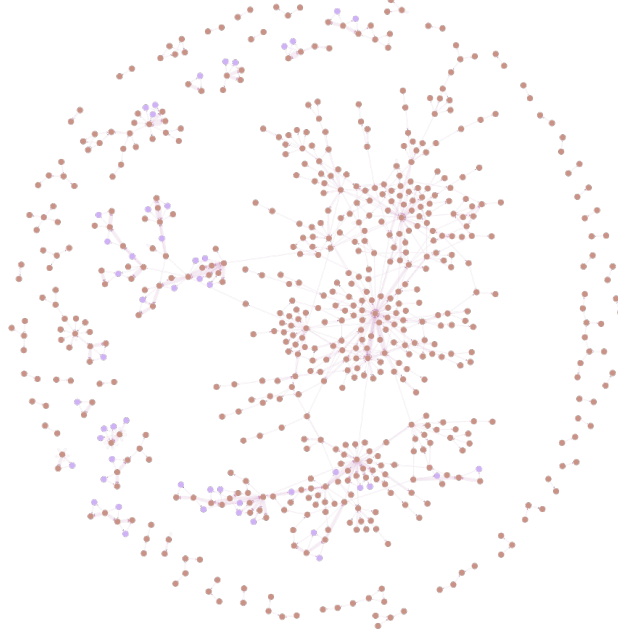


Figure 3.1: This graph consists combination of nodes and their relations extracted from 2 different judgment summaries, 1114158_extracted.txt and 1316559_extracted.txt

Chapter 4

Conclusion and Future Work

In this report, we have compared three LLMs for the Triplet Generation task by using the EDC technique introduced by [16]. We found *LLama3.1-8b-Instruct* along with Self-Canonicalization method to be the capable model for extracting the best quality triplets corresponding to the Legal Domain.

During our comparison, we found that LLama 3.1 (Self-Canonicalization) was taking less time (8.9 hrs.) compared to the Mistral (26.7 hrs.), LLama 2 (22.5 hrs.) and LLama 3.1 (Target Alignment) (9.6 hrs.).

Talking about the quality of triplets extracted, LLama 3.1 (Self-Canonicalization) extracted the maximum number of valid triplets, i.e., 666, in comparison of Mistral (235), LLama 2 (72) and LLama 3.1 (Target Alignment) (34). Also, the credit here goes to the Self-Canonicalization approach, which was the key factor behind the good quality of triplets generated.

Till now we have only utilised one pipeline and that leaves us with its drawbacks too. The major drawback of using the EDC technique is the Inference time it takes to Extract, Define and then Canonicalize the Triplets. That is why this approach is not much scalable to extract triplets of large number of judgments and other legal documents. In the future we aim to explore more pipelines and compare both the inference time and the quality of triplets generated.

In the future work, we aim to develop a novel approach for Information extraction and knowledge graph construction especially for legal domain that addresses the limitations of EDC. Our main objective would be to harness the power of pretrained LLMs and prompt engineering to produce better quality triplets.

We also aim to build the largest Knowledge Graph of Indian legal documents,

providing legal professionals and students with access to a comprehensive database that enhances their everyday work while promoting contributions to open-source projects and giving back to the community.

References

- [1] D. Xu, W. Chen, W. Peng, C. Zhang, T. Xu, X. Zhao, X. Wu, Y. Zheng, Y. Wang, and E. Chen, “Large language models for generative information extraction: A survey,” 2024.
- [2] M. M. Akter and M.-M. Rahoman, “A systematic review on knowledge graphs classification and their various usages,” *Artificial Intelligence Evolution*, no date. Accessed: 27 December 2024.
- [3] E. Riloff and R. Jones, “Learning dictionaries for information extraction by multi-level bootstrapping,” pp. 474–479, 01 1999.
- [4] H. Zhong, C. Xiao, C. Tu, T. Zhang, Z. Liu, and M. Sun, “How does NLP benefit legal system: A summary of legal artificial intelligence,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, eds.), (Online), pp. 5218–5230, Association for Computational Linguistics, July 2020.
- [5] D. Xu, W. Chen, W. Peng, C. Zhang, T. Xu, X. Zhao, X. Wu, Y. Zheng, and E. Chen, “Large language models for generative information extraction: A survey,” *arXiv preprint arXiv:2312.17617*, 2023.
- [6] E. Agichtein and L. Gravano, “Snowball: extracting relations from large plain-text collections,” in *Proceedings of the Fifth ACM Conference on Digital Libraries*, DL ’00, (New York, NY, USA), p. 85–94, Association for Computing Machinery, 2000.
- [7] L. Page, S. Brin, R. Motwani, and T. Winograd, “The pagerank citation ranking : Bringing order to the web,” in *The Web Conference*, 1999.
- [8] A. Yates, M. Banko, M. Broadhead, M. Cafarella, O. Etzioni, and S. Soderland, “TextRunner: Open information extraction on the web,” in *Proceedings of Human Language Technologies: The Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)* (B. Carpenter,

- A. Stent, and J. D. Williams, eds.), (Rochester, New York, USA), pp. 25–26, Association for Computational Linguistics, Apr. 2007.
- [9] F. Wu and D. S. Weld, “Open information extraction using Wikipedia,” in *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics* (J. Hajič, S. Carberry, S. Clark, and J. Nivre, eds.), (Uppsala, Sweden), pp. 118–127, Association for Computational Linguistics, July 2010.
 - [10] Mausam, M. Schmitz, S. Soderland, R. Bart, and O. Etzioni, “Open language learning for information extraction,” in *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning* (J. Tsujii, J. Henderson, and M. Paşca, eds.), (Jeju Island, Korea), pp. 523–534, Association for Computational Linguistics, July 2012.
 - [11] K. Gashteovski, R. Gemulla, and L. del Corro, “MinIE: Minimizing facts in open information extraction,” in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing* (M. Palmer, R. Hwa, and S. Riedel, eds.), (Copenhagen, Denmark), pp. 2630–2640, Association for Computational Linguistics, Sept. 2017.
 - [12] C. Niklaus, M. Cetto, A. Freitas, and S. Handschuh, “A survey on open information extraction,” in *Proceedings of the 27th International Conference on Computational Linguistics* (E. M. Bender, L. Derczynski, and P. Isabelle, eds.), (Santa Fe, New Mexico, USA), pp. 3866–3878, Association for Computational Linguistics, Aug. 2018.
 - [13] X. Wei, X. Cui, N. Cheng, X. Wang, X. Zhang, S. Huang, P. Xie, J. Xu, Y. Chen, M. Zhang, Y. Jiang, and W. Han, “Chatie: Zero-shot information extraction via chatting with chatgpt,” 2024.
 - [14] Z. Bi, J. Chen, Y. Jiang, F. Xiong, W. Guo, H. Chen, and N. Zhang, “Codekgc: Code language model for generative knowledge graph construction,” 2024.
 - [15] S. Wadhwa, S. Amir, and B. Wallace, “Revisiting relation extraction in the era of large language models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (A. Rogers, J. Boyd-Graber, and N. Okazaki, eds.), (Toronto, Canada), pp. 15566–15589, Association for Computational Linguistics, July 2023.
 - [16] B. Zhang and H. Soh, “Extract, define, canonicalize: An llm-based framework for knowledge graph construction,” *arXiv preprint arXiv:2404.03868*, 2024.
 - [17] P.-L. Huguet Cabot and R. Navigli, “REBEL: Relation extraction by end-to-end language generation,” in *Findings of the Association for Computational Linguistics*:

EMNLP 2021 (M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, eds.), (Punta Cana, Dominican Republic), pp. 2370–2381, Association for Computational Linguistics, Nov. 2021.

Appendix A

Sentence level comparison, Mistral (7b) vs LLama3.1 (8b)

If we ignore the triplet generation time and focus only on the quality of triplets extracted by the models, we can say from the experiments that *Mistral-7B-Instruct-v0.2 (Target-Alignment)* and *LLama3.1-8b-Instruct (Self-Canonicalization)* extracted the best triplets among all. To compare both the models sentence wise we created a script and generated the text file containing couples of triplet set.

Refer to 6 randomly selected sample sentences and their respective extracted triplets from the Judgment, shown in Table A.1 and Table A.2. The text in **red colour** is the hallucinated text generated by Mistral (7b).

Sentence	Mistral 7b	LLama 3.1
The first respondent hypothecated its two Fokker F27-500 Aircrafts, bearing Registration No. VT-NEJ (12684) and VT-NEK (10687) to the appellant under Deed of Hypothecation dated 151997, to secure the outstanding amounts.	[[‘appellant’, ‘inflows’, ‘Fokker F27-500 Aircraft (VT-NEJ)’], [‘appellant’, ‘inflows’, ‘Fokker F27-500 Aircraft (VT-NEK)’]]	[[‘Fokker F27-500 Aircraft’, ‘model’, ‘F27-500’], [‘Fokker F27-500 Aircraft’, ‘registration number’, ‘VT-NEJ’], [‘Fokker F27-500 Aircraft’, ‘registration number’, ‘VT-NEK’], [‘VT-NEJ’, ‘registration number’, ‘12684’], [‘VT-NEK’, ‘registration number’, ‘10687’], [‘Fokker F27-500 Aircraft’, ‘hypothecated to’, ‘appellant’], [‘appellant’, ‘received’, ‘Deed of Hypothecation’], [‘Deed of Hypothecation’, ‘date’, ‘151997’]]
Under clause (6), NEPC India declared that it would not assign, sell, pledge, charge, underlet or otherwise encumber or part with the possession, custody or beneficial interest in respect of the two aircrafts without the previous written consent of IOC.	[[‘NEPC India’, ‘owner of’, ‘two aircrafts’]]	[[‘NEPC India’, ‘declared’, ‘clause (6)’], [‘NEPC India’, ‘would not assign’, ‘aircrafts’], [‘NEPC India’, ‘would not sell’, ‘aircrafts’], [‘NEPC India’, ‘would not pledge’, ‘aircrafts’], [‘NEPC India’, ‘would not charge’, ‘aircrafts’], [‘NEPC India’, ‘would not underlet’, ‘aircrafts’], [‘NEPC India’, ‘would not encumber’, ‘aircrafts’], [‘NEPC India’, ‘would not part with possession’, ‘aircrafts’], [‘NEPC India’, ‘would not part with custody’, ‘aircrafts’], [‘NEPC India’, ‘would not part with beneficial interest’, ‘aircrafts’], [‘aircrafts’, ‘require consent of’, ‘IOC’]]
As NEPC India failed to pay the first two installments as per schedule, IOC stopped supply of aircraft fuel on 361997.	[]	[[‘NEPC India’, ‘failed to pay’, ‘first two installments’], [‘first two installments’, ‘due date’, ‘schedule’], [‘schedule’, ‘related to’, ‘NEPC India’], [‘NEPC India’, ‘entity affected by’, ‘IOC’], [‘IOC’, ‘action taken’, ‘stopped supply of aircraft fuel’], [‘stopped supply of aircraft fuel’, ‘date’, ‘361997’]]

Table A.2: Sentence level comparison of the triplets extracted by two popular models

Appendix B

Experiments Output Files

The datasets and the generated triplet files of the project are available on GitHub ¹. The repository has been organised as follows:

- **REBEL/**
 - **19 Sentences/** contains the dataset and triplets generated on 19 Sentences
 - **100 Sentences/** contains the dataset and triplets generated on 100 Sentences
 - **1000 Sentences/** contains the dataset and triplets generated on 1000 Sentences
- **The Judgement & Experiments/**
 - **The Judgment Dataset/** contains the Judgment dataset, both raw and tokenized.
 - **Models & Outputs/** contains the Triplets generated using 3 different LLMs.
- **10 Judgment Summaries/**
 - **Summaries/** contains preprocessed summaries of 10 different Judgments.
 - **Output/** contains the triplets generated corresponding to the summaries using LLama3.1-8b (self-canonicalization).

All the above folder contains one extra file “Combined File” consisting of original sentences attached with generated triplets which is helpful for Analysis.

Along with the Judgment and 10 summaries, we have also provided the csv files which contains the triplets, each marked as valid or Invalid.

¹<https://github.com/Harsh-Yadav-02/LegalKG>